CASE STUDY 1: UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement

To check whether a student is eligible for a scholarship based on marks, attendance, and annual family income.

2. Objectives

To determine scholarship eligibility using conditional statements.

3. Assumptions

- Minimum aggregate marks required are 75%.
- Minimum attendance required is 75%.
- Annual family income must not exceed Rs. 2,00,000.

4. Algorithm

1. Start.
2. Read marks, attendance, and annual family income.
3. Check all eligibility conditions.
4. If all conditions are satisfied, display "Eligible for scholarship".
5. Otherwise, display "Not eligible for scholarship".
6. Stop.

5. Flowchart

Start -> Input details -> Check eligibility conditions -> Display result -> Stop.

6. Pseudocode

```
START
READ marks, attendance, income

IF marks >= 75 AND attendance >= 75 AND income <= 200000
   PRINT "Eligible for scholarship"
ELSE
   PRINT "Not eligible for scholarship"
END IF

STOP
```

7. Python Source Code

```
marks = int(input("Enter aggregate marks: "))
attendance = int(input("Enter attendance percentage: "))
income = int(input("Enter annual family income: "))

if marks >= 75 and attendance >= 75 and income <= 200000:
   print("Eligible for scholarship")
else:
   print("Not eligible for scholarship")
```

8. Test Cases

Marks: 85, Attendance: 80, Income: 150000

Expected Output: Eligible

Marks: 70, Attendance: 80, Income: 150000

Expected Output: Not eligible

Marks: 90, Attendance: 70, Income: 100000

Expected Output: Not eligible

Marks: 90, Attendance: 90, Income: 250000

Expected Output: Not eligible

9. Output

```
Enter student name: Akhila
Enter aggregate marks: 90
Enter subject marks: 80
Enter attendance: 90
Enter family income: 80000
Akhila is eligible for scholarship
```

10. Conclusion

The program checks scholarship eligibility using marks, attendance, family income, and conditional statements.

CASE STUDY 2: PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

To calculate an electricity bill based on the number of units consumed by a household.

2. Objectives

To calculate electricity charges using different unit rates and a fixed charge.

3. Assumptions

- First 100 units cost Rs. 2 per unit.
- Units from 101 to 200 cost Rs. 3 per unit.
- Units above 200 cost Rs. 5 per unit.
- A fixed charge of Rs. 50 is added.

4. Algorithm

1. Start.
2. Read the number of units consumed.
3. Check the unit range.
4. Calculate the electricity charge.
5. Add the fixed charge.
6. Display the total bill.
7. Stop.

5. Flowchart

Start -> Input units -> Check unit range -> Calculate charge -> Add fixed charge -> Display bill -> Stop.

6. Pseudocode

```
START
READ units
IF units <= 100
   bill = units * 2
ELSE IF units <= 200
   bill = 200 + (units - 100) * 3
ELSE
   bill = 500 + (units - 200) * 5
END IF
bill = bill + 50
PRINT bill
STOP
```

7. Python Source Code

```
units = int(input("Enter units consumed: "))

if units <= 100:
   bill = units * 2
elif units <= 200:
   bill = 200 + (units - 100) * 3
```

```
else:
    bill = 500 + (units - 200) * 5

bill = bill + 50

print("Total electricity bill: Rs.", bill)
```

8. Test Cases

Units: 80 -> Bill: Rs. 210

Units: 150 -> Bill: Rs. 400

Units: 250 -> Bill: Rs. 850

9. Output

```
Enter electricity units: 150
Electricity Bill: 400
```

10. Conclusion

The program calculates electricity charges using unit-based rates and adds a fixed charge.

CASE STUDY 3: HOSPITAL BILLING AND PATIENT CLASSIFICATION

1. Problem Statement

To calculate a patient's hospital bill and classify it based on the final bill amount.

2. Objectives

To calculate hospital charges, apply a discount, and classify the patient's bill.

3. Assumptions

- The bill includes consultation, diagnosis, and treatment charges.
- A 10% discount is provided when the total bill is Rs. 4,000 or more.
- A final bill of Rs. 3,000 or more is classified as a high bill.

4. Algorithm

1. Start.
2. Read the patient's name and hospital charges.
3. Calculate the total bill.
4. Check whether the total is at least Rs. 4,000.
5. Apply a 10% discount if eligible.
6. Calculate the final bill.
7. Classify the bill as high or normal.
8. Stop.

5. Flowchart

Start -> Input patient details -> Calculate total -> Check discount -> Calculate final bill -> Classify bill -> Display result -> Stop.

6. Pseudocode

```
START
READ name, consultation, diagnosis, treatment
total = consultation + diagnosis + treatment

IF total >= 4000
   discount = total * 10 / 100
ELSE
   discount = 0
END IF

final_bill = total - discount
PRINT total, discount, final_bill

IF final_bill >= 3000
   PRINT "High Bill"
ELSE
   PRINT "Normal Bill"
END IF
STOP
```

7. Python Source Code

```
name = input("Enter patient name: ")
consultation = int(input("Enter consultation charge: "))
diagnosis = int(input("Enter diagnosis charge: "))
treatment = int(input("Enter treatment charge: "))

total = consultation + diagnosis + treatment

if total >= 4000:
    discount = total * 10 / 100
else:
    discount = 0

final_bill = total - discount

print("Patient name:", name)
print("Total bill:", total)
print("Discount:", discount)
print("Final bill:", final_bill)

if final_bill >= 3000:
    print("High Bill")
else:
    print("Normal Bill")
```

8. Test Cases

Consultation: 500, Diagnosis: 1500, Treatment: 2500 -> High Bill

Consultation: 500, Diagnosis: 500, Treatment: 1000 -> Normal Bill

Consultation: 1000, Diagnosis: 1500, Treatment: 2000 -> High Bill

9. Output

```
Enter patient name: Akhila
Enter consultation charges: 500
Enter diagnosis charges: 1500
Enter treatment charges: 2500
Patient Name: Akhila
Total Bill: 4500
Discount: 450.0
Final Bill: 4050.0
High Bill

Process finished with exit code 0
```

10. Conclusion

The program calculates hospital charges, applies a discount, and classifies the final bill using conditional statements.

CASE STUDY 4: EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement

To calculate an employee's gross salary, salary deductions, and net salary.

2. Objectives

To calculate salary and deductions using arithmetic and conditional statements.

3. Assumptions

- Gross salary includes basic salary, allowance, overtime, and incentive.
- Deduction is 5% for gross salary up to Rs. 20,000.
- Deduction is 10% for gross salary above Rs. 20,000 and up to Rs. 30,000.
- Deduction is 15% for gross salary above Rs. 30,000.

4. Algorithm

1. Start.
2. Read employee details and salary components.
3. Calculate gross salary.
4. Determine the deduction percentage.
5. Calculate deduction and net salary.
6. Display the results.
7. Stop.

5. Flowchart

Start -> Input salary details -> Calculate gross salary -> Check salary range -> Calculate deduction -> Calculate net salary -> Display results -> Stop.

6. Pseudocode

START

READ name, basic, allowance, overtime, incentive

gross = basic + allowance + overtime + incentive

```
IF gross <= 20000
   deduction = gross * 5 / 100
ELSE IF gross <= 30000
   deduction = gross * 10 / 100
ELSE
   deduction = gross * 15 / 100
END IF

net = gross - deduction
PRINT gross, deduction, net
STOP
```

7. Python Source Code

```
name = input("Enter employee name: ")
basic = int(input("Enter basic salary: "))
allowance = int(input("Enter allowance: "))
overtime = int(input("Enter overtime amount: "))
incentive = int(input("Enter incentive: "))

gross = basic + allowance + overtime + incentive

if gross <= 20000:
   deduction = gross * 5 / 100
elif gross <= 30000:
   deduction = gross * 10 / 100
else:
```

```
deduction = gross * 15 / 100

net = gross - deduction

print("Employee name:", name)
print("Gross salary:", gross)
print("Deduction:", deduction)
print("Net salary:", net)
```

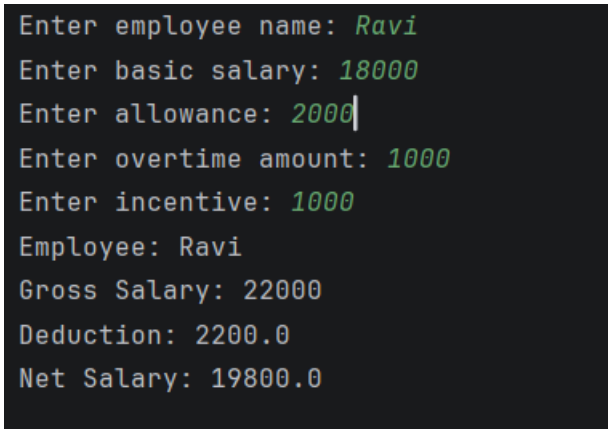
8. Test Cases

Gross salary: Rs. 20000 -> Deduction: Rs. 1000 -> Net: Rs. 19000

Gross salary: Rs. 22000 -> Deduction: Rs. 2200 -> Net: Rs. 19800

Gross salary: Rs. 35000 -> Deduction: Rs. 5250 -> Net: Rs. 29750

9. Output



10. Conclusion

The program calculates gross salary, deductions, and net salary using arithmetic and conditional statements.

CASE STUDY 5: MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

1. Problem Statement

To select a suitable mobile plan based on the customer's budget, daily data usage, and required validity.

2. Objectives

To compare customer requirements with available plans and display a suitable option.

3. Assumptions

- Plan A costs Rs. 199, provides up to 2 GB per day, and has 28-day validity.
- Plan B costs Rs. 299, provides up to 3 GB per day, and has 56-day validity.
- Plan C costs Rs. 399, provides up to 4 GB per day, and has 84-day validity.
- These are illustrative values for the program.

4. Algorithm

1. Start.
2. Read budget, daily data usage, and required validity.
3. Check whether Plan A is suitable.
4. If not, check Plan B and then Plan C.
5. Display the suitable plan or a message if no plan matches.
6. Stop.

5. Flowchart

Start -> Input budget, data, and validity -> Check Plan A -> Check Plan B -> Check Plan C -> Display result -> Stop.

6. Pseudocode

```
START
READ budget, data, days

IF budget >= 199 AND data <= 2 AND days <= 28
   PRINT "Plan A"
ELSE IF budget >= 299 AND data <= 3 AND days <= 56
   PRINT "Plan B"
ELSE IF budget >= 399 AND data <= 4 AND days <= 84
   PRINT "Plan C"
ELSE
   PRINT "No suitable plan"
END IF

STOP
```

7. Python Source Code

```
budget = int(input("Enter your budget: "))
data = int(input("Enter daily data usage in GB: "))
days = int(input("Enter required validity in days: "))
```

```
if budget >= 199 and data <= 2 and days <= 28:
    print("Suitable plan: Plan A - Rs. 199")
elif budget >= 299 and data <= 3 and days <= 56:
    print("Suitable plan: Plan B - Rs. 299")
elif budget >= 399 and data <= 4 and days <= 84:
    print("Suitable plan: Plan C - Rs. 399")
else:
    print("No suitable plan available")
```

8. Test Cases

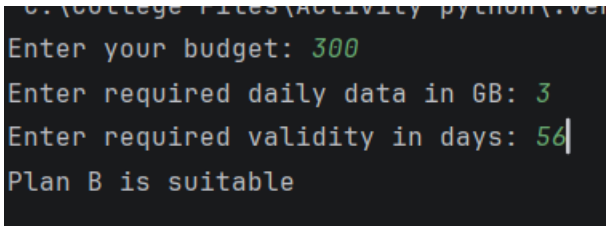
Budget: 200, Data: 2 GB, Validity: 28 days -> Plan A

Budget: 300, Data: 3 GB, Validity: 56 days -> Plan B

Budget: 400, Data: 4 GB, Validity: 84 days -> Plan C

Budget: 150, Data: 2 GB, Validity: 28 days -> No suitable plan

9. Output



10. Conclusion

The program compares customer requirements with available plans and displays a suitable mobile plan.

CASE STUDY 6: RAILWAY FARE AND RESERVATION CALCULATOR

1. Problem Statement

To calculate railway fare based on passenger age and travel distance, including a reservation fee.

2. Objectives

To calculate the basic fare, apply age-based concessions, and display the final fare.

3. Assumptions

- Up to 100 km: Rs. 1 per km.
- Above 100 km and up to 300 km: Rs. 1.50 per km.
- Above 300 km: Rs. 2 per km.
- Passengers aged 60 or above receive a 20% concession.
- Children aged 12 or below receive a 50% concession.
- Reservation fee is Rs. 30.
- The selected distance rate applies to the entire distance.

4. Algorithm

1. Start.
2. Read passenger name, age, and distance.
3. Calculate fare according to distance.
4. Apply an age concession if eligible.
5. Add the reservation fee.
6. Display the final fare.

7. Stop.

5. Flowchart

Start -> Input passenger details -> Calculate basic fare -> Check age -> Apply concession -> Add reservation fee -> Display final fare -> Stop.

6. Pseudocode

```
START
READ name, age, distance

IF distance <= 100
   fare = distance * 1
ELSE IF distance <= 300
   fare = distance * 1.5
ELSE
   fare = distance * 2
END IF

IF age >= 60
   fare = fare * 80 / 100
ELSE IF age <= 12
   fare = fare * 50 / 100
END IF

final_fare = fare + 30
PRINT final_fare
```

STOP

7. Python Source Code

```
name = input("Enter passenger name: ")
age = int(input("Enter passenger age: "))
distance = int(input("Enter distance in km: "))

if distance <= 100:
   fare = distance * 1
elif distance <= 300:
   fare = distance * 1.5
else:
   fare = distance * 2

if age >= 60:
   fare = fare * 80 / 100
elif age <= 12:
   fare = fare * 50 / 100

final_fare = fare + 30

print("Passenger name:", name)
print("Final railway fare: Rs.", final_fare)
```

8. Test Cases

Age: 25, Distance: 80 km -> Fare: Rs. 110

Age: 25, Distance: 150 km -> Fare: Rs. 255

Age: 65, Distance: 100 km -> Fare: Rs. 110

Age: 10, Distance: 100 km -> Fare: Rs. 80

9. Output

```
Enter passenger name: Ravi
Enter passenger age: 25
Enter distance in km: 150
Passenger: Ravi
Basic Fare: 225.0
Reservation Fee: 30
Total Fare: 255.0
```

10. Conclusion

The program calculates railway fare using distance-based rates, age concessions, and a reservation fee.

CASE STUDY 7: RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

To calculate a retail bill using product prices, quantities, discounts, and tax.

2. Objectives

To calculate the subtotal, apply a discount, calculate tax, and display the final bill.

3. Assumptions

- The customer purchases two products.

- A 10% discount applies when the subtotal is Rs. 1,000 or more.

- Tax is 5% of the amount after discount.

4. Algorithm

1. Start.

2. Read product prices and quantities.

3. Calculate the subtotal.

4. Check whether the discount applies.

5. Subtract the discount.

6. Calculate tax.

7. Display the final bill.

8. Stop.

5. Flowchart

Start -> Input product prices and quantities -> Calculate subtotal -> Check discount -> Calculate amount after discount -> Calculate tax -> Display final bill -> Stop.

6. Pseudocode

START

READ price1, quantity1, price2, quantity2

total = price1 * quantity1 + price2 * quantity2

```
IF total >= 1000
   discount = total * 10 / 100
ELSE
   discount = 0
END IF

amount = total - discount
tax = amount * 5 / 100
final_bill = amount + tax

PRINT total, discount, tax, final_bill
STOP
```

7. Python Source Code

```
price1 = int(input("Enter first product price: "))
quantity1 = int(input("Enter first product quantity: "))
price2 = int(input("Enter second product price: "))
quantity2 = int(input("Enter second product quantity: "))

total = price1 * quantity1 + price2 * quantity2

if total >= 1000:
   discount = total * 10 / 100
else:
   discount = 0

amount = total - discount
```

```
tax = amount * 5 / 100
final_bill = amount + tax

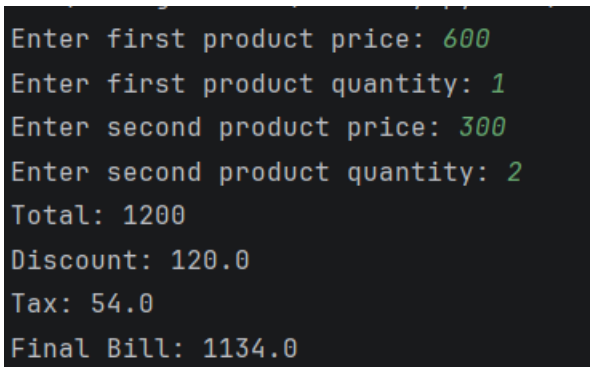
print("Subtotal:", total)
print("Discount:", discount)
print("Tax:", tax)
print("Final bill: Rs.", final_bill)
```

8. Test Cases

Price1: 600, Quantity1: 1, Price2: 300, Quantity2: 2 -> Final bill: Rs. 1134.0

Price1: 200, Quantity1: 1, Price2: 100, Quantity2: 1 -> Final bill: Rs. 315.0

9. Output



10. Conclusion

The program calculates a retail bill by applying the discount and tax according to the given conditions.

CASE STUDY 8: MANUFACTURING COST AND PROFITABILITY ANALYSIS

1. Problem Statement

To calculate the total manufacturing cost and determine whether the profit target is achieved.

2. Objectives

To calculate production costs, find profit, and check the profit target.

3. Assumptions

- Total cost includes material, labour, and machine costs.
- The profit target is Rs. 2,000.
- All amounts refer to the same production batch.

4. Algorithm

1. Start.
2. Read material cost, labour cost, machine cost, and selling price.
3. Calculate total manufacturing cost.
4. Calculate profit.
5. Check whether profit is at least Rs. 2,000.
6. Display the results.
7. Stop.

5. Flowchart

Start -> Input costs and selling price -> Calculate total cost -> Calculate profit -> Check profit target -> Display result -> Stop.

6. Pseudocode

```
START
READ material, labour, machine, selling_price

total_cost = material + labour + machine
profit = selling_price - total_cost

PRINT total_cost, profit

IF profit >= 2000
   PRINT "Profit target achieved"
ELSE
   PRINT "Profit target not achieved"
END IF

STOP
```

7. Python Source Code

```
material = int(input("Enter material cost: "))
labour = int(input("Enter labour cost: "))
machine = int(input("Enter machine cost: "))
selling_price = int(input("Enter selling price: "))
```

```
total_cost = material + labour + machine

profit = selling_price - total_cost


print("Total manufacturing cost:", total_cost)

print("Profit:", profit)


if profit >= 2000:

   print("Profit target achieved")

else:

   print("Profit target not achieved")
```

8. Test Cases

Material: 5000, Labour: 2000, Machine: 1000, Selling price: 12000 -> Profit: 4000

Material: 5000, Labour: 2000, Machine: 1000, Selling price: 8500 -> Profit: 500

9. Output

```
Enter material cost: 5000
Enter labour cost: 2000
Enter machine cost: 1000
Enter selling price: 12000
Total Cost: 8000
Profit: 4000
Profit Target Achieved
```

10. Conclusion

The program calculates manufacturing cost and profit and checks whether the target is achieved.

CASE STUDY 9: MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. Problem Statement

To convert travel expenses from Indian rupees into another currency and check whether the total fits within a given budget.

2. Objectives

To convert expenses, calculate the total with an additional fee, and compare the total with the budget.

3. Assumptions

- The example exchange rate is Rs. 1 = 0.011 units of the target currency.
- An additional 2% fee is applied to converted expenses.
- The budget is 300 units of the target currency.
- The exchange rate and budget are illustrative values, not live exchange information.

4. Algorithm

1. Start.
2. Read food, hotel, and transport expenses in rupees.
3. Convert each expense using the given exchange rate.
4. Calculate the total and additional fee.
5. Identify the largest expense category.
6. Check whether the total is within budget.
7. Display the results.

8. Stop.

5. Flowchart

Start -> Input expenses -> Convert expenses -> Calculate total and fee -> Identify largest expense -> Check budget -> Display results -> Stop.

6. Pseudocode

```
START
READ food, hotel, transport

food_cost = food * 0.011
hotel_cost = hotel * 0.011
transport_cost = transport * 0.011

total = food_cost + hotel_cost + transport_cost
fee = total * 2 / 100
final_total = total + fee

PRINT converted expenses and final_total

IF food_cost >= hotel_cost AND food_cost >= transport_cost
   PRINT "Food is the largest expense"
ELSE IF hotel_cost >= transport_cost
   PRINT "Hotel is the largest expense"
ELSE
   PRINT "Transport is the largest expense"
```

```
END IF

IF final_total <= 300
   PRINT "Within budget"
ELSE
   PRINT "Over budget"
END IF

STOP
```

7. Python Source Code

```
food = int(input("Enter food expenses in INR: "))
hotel = int(input("Enter hotel expenses in INR: "))
transport = int(input("Enter transport expenses in INR: "))

food_cost = food * 0.011
hotel_cost = hotel * 0.011
transport_cost = transport * 0.011

total = food_cost + hotel_cost + transport_cost
fee = total * 2 / 100
final_total = total + fee

print("Food expense:", round(food_cost, 2))
print("Hotel expense:", round(hotel_cost, 2))
print("Transport expense:", round(transport_cost, 2))
print("Total with fee:", round(final_total, 2))
```

```
if food_cost >= hotel_cost and food_cost >= transport_cost:
    print("Food is the largest expense")
elif hotel_cost >= transport_cost:
    print("Hotel is the largest expense")
else:
    print("Transport is the largest expense")

if final_total <= 300:
    print("Within budget")
else:
    print("Over budget")
```

8. Test Cases

Food: 5000, Hotel: 15000, Transport: 5000 -> Total: 280.5 -> Within budget

Food: 10000, Hotel: 20000, Transport: 10000 -> Total: 448.8 -> Over budget

9. Output

```
Enter food expense in rupees: 5000
Enter hotel expense in rupees: 15000
Enter transport expense in rupees: 5000
Food Cost: 55.0
Hotel Cost: 165.0
Transport Cost: 55.0
Total Cost with Fee: 280.5
Largest Expense: Hotel
Within Budget
```

10. Conclusion

The program converts travel expenses, calculates the total including a fee, identifies the largest expense, and checks the budget.

CASE STUDY 10: CAMPUS EVENT FINANCIAL PLANNING

1. Problem Statement

To calculate the cost and income of a campus event and determine whether the event makes a profit or has a shortfall.

2. Objectives

To calculate event expenses, registration income, sponsorship, and the final balance.

3. Assumptions

- Venue cost is Rs. 10,000.
- Equipment cost is Rs. 5,000.
- Transport cost is Rs. 3,000.
- Catering costs Rs. 150 per participant.
- Certificates cost Rs. 20 per participant.
- Registration income depends on the number of participants and the fee per participant.

4. Algorithm

1. Start.
2. Read the number of participants, registration fee, and sponsorship.
3. Calculate catering and certificate costs.

4. Calculate the total event cost.

5. Calculate registration income and total income.

6. Calculate the balance.

7. Display whether the event is profitable or has a shortfall.

8. Stop.

5. Flowchart

Start -> Input participants, fee, and sponsorship -> Calculate event costs -> Calculate total income -> Calculate balance -> Check balance -> Display result -> Stop.

6. Pseudocode

START

READ participants, fee, sponsorship

venue = 10000

equipment = 5000

transport = 3000

catering = participants * 150

certificates = participants * 20

total_cost = venue + equipment + transport + catering + certificates

registration = participants * fee

income = registration + sponsorship

balance = income - total_cost

PRINT total_cost, income, balance

```
IF balance >= 0
   PRINT "Event is profitable"
ELSE
   PRINT "Event has a shortfall"
END IF

STOP
```

7. Python Source Code

```
participants = int(input("Enter number of participants: "))
fee = int(input("Enter registration fee per participant: "))
sponsorship = int(input("Enter sponsorship amount: "))

venue = 10000
equipment = 5000
transport = 3000
catering = participants * 150
certificates = participants * 20

total_cost = venue + equipment + transport + catering + certificates

registration = participants * fee
income = registration + sponsorship

balance = income - total_cost
```

```
print("Total event cost:", total_cost)
print("Total income:", income)
print("Balance:", balance)

if balance >= 0:
    print("Event is profitable")
else:
    print("Event has a shortfall")
```

8. Test Cases

Participants: 100, Fee: 300, Sponsorship: 10000 -> Balance: Rs. 5000 -> Profitable

Participants: 100, Fee: 200, Sponsorship: 5000 -> Balance: Rs. -10000 -> Shortfall

9. Output

```
Enter number of participants: 100
Enter registration fee per person: 300
Enter sponsorship amount: 10000
Total Cost: 35000
Registration Income: 30000
Sponsorship: 10000
Total Income: 40000
Final Balance: 5000
Event is Profitable
```

10. Conclusion

The program calculates event expenses and income and determines whether the event is profitable.